TWINEVAL AI

AI-Assisted Academic Evaluation and Learning Analytics Platform

CURRENT DEPLOYED SYSTEM

Complete Technical Project Report and Viva Guide

Prepared by

Team Member	Register Number
ASHWIN KIRBAA B	714025104028
AKHILESHKAR T P	714025104011
ALEN K R	714025104017
ASWIN R	714025104030

Bachelor of Engineering - Computer Science and Engineering

Sri Shakthi Institute of Engineering and Technology, Coimbatore

Revision 2.0 | 31 October 2026 | Repository commit 6e587f4

Document Control and Revision Summary

Item	Current value
Document purpose	Explain the complete deployed TwinEval system for review, demonstration, maintenance, and viva examination.
Current frontend	Vercel static deployment: https://twineval-ai.vercel.app
Current backend	Render Node/Express API: https://twineval-api.onrender.com
Production data	Supabase PostgreSQL plus Supabase Storage
AI execution	Local Python Flask service and local Ollama on each client computer
Primary local models	qwen3:4b-instruct for text; gemma3:4b for OCR/vision
Authentication	Gmail-only password signup with email verification, plus Google Identity Services sign-in
Source baseline	GitHub main branch, commit 6e587f4

What changed since the old report

The earlier report described a mainly local Node + SQLite implementation and treated several functions as unverified. The current application is deployed, uses a Vercel/Render/Supabase architecture, supports Google Sign-In and email verification, stores files persistently, runs AI locally from the browser, includes teacher review and student feedback, and has working analytics and report export paths.

Table of Contents

Document Control and Revision Summary 2

Table of Contents 3

1. Executive Summary 7

2. Problem Definition, Objectives, and Scope 8

2.1 Problem addressed . . . 8

2.2 Core objectives . . . 8

2.3 Scope boundaries . . . 8

3. Project Evolution: Old Report vs Current System 9

4. Users, Roles, and Use Cases 10

4.1 Student use cases . . . 10

4.2 Teacher use cases . . . 10

5. Current System Architecture 11

5.1 Why the architecture is hybrid . . . 11

5.2 Component responsibilities . . . 12

6. Technology Stack and Why Each Technology Is Used 13

7. End-to-End Functional Workflow 14

7.1 Question paper workflow . . . 14

7.2 Student evaluation workflow . . . 14

8. Authentication, Account Lifecycle, and Profiles 15

8.1 Password-based Gmail registration . . . 15

8.2 Continue with Google . . . 15

8.3 Session and profile behavior . . . 15

9. AI Engine and Evaluation Intelligence 16

9.1 Local service design . . . 16

9.2 Document extraction . . . 16

9.3 Question and answer segmentation . . . 16

9.4 Answer evaluation . . . 16

9.5 Result object . . . 17

10. Scoring, Grades, Review, and Final Marks 18
10.1 Grade scale . . . 18
10.2 Overall calculation . . . 18
10.3 Teacher review . . . 18
10.4 Analytics calculations . . . 19

11. Database Design and Data Model 20
11.1 Dual database adapter . . . 20
11.2 Table dictionary . . . 21
11.3 Important states . . . 21

12. REST API Design 22

13. Frontend Application 23
13.1 Page map . . . 23
13.2 Shared client behavior . . . 23

14. File Handling, Storage, and Report Generation 24
14.1 Upload policy . . . 24
14.2 Evaluation report PDF . . . 24
14.3 CSV analytics . . . 24

15. Current Application Screens 25

16. Deployment Architecture and Configuration 28
16.1 Production deployment . . . 28
16.2 Important environment variables . . . 28
16.3 Why secrets must stay server-side . . . 29

17. Installation and Execution Guide 30
17.1 Local prerequisites . . . 30
17.2 Windows startup . . . 30
17.3 Manual commands . . . 30

18. Security and Privacy Assessment 31
18.1 Implemented controls . . . 31
18.2 Current risks and recommended hardening . . . 31

19. Testing and Validation Strategy 32
19.1 Functional test matrix . . . 32
19.2 Evaluation quality testing . . . 32

20. Current Results and Demonstrated Capabilities 33

21. Limitations and Engineering Trade-offs 34

22. Future Enhancements 35

23. Current Repository Structure 36

24. Troubleshooting Guide 37

25. Viva and Demonstration Question Bank 38

Q: What is TwinEval AI? . . . 38
Q: What is the main innovation? . . . 38
Q: Why is it called hybrid? . . . 38
Q: Why not run Ollama on Render? . . . 38
Q: Why Vercel? . . . 38
Q: Why Render? . . . 38
Q: Why Supabase? . . . 38
Q: Why keep SQLite support? . . . 38
Q: What is JWT? . . . 38
Q: How are passwords protected? . . . 38
Q: How does Google Sign-In work? . . . 38
Q: Why is SMTP still needed? . . . 39
Q: How are uploads protected? . . . 39
Q: How does PDF OCR work? . . . 39
Q: Why use two models? . . . 39
Q: How are questions detected? . . . 39
Q: How are objective answers graded? . . . 39
Q: How are descriptive answers graded? . . . 39
Q: What is semantic similarity? . . . 39
Q: What is Bloom's taxonomy used for? . . . 39
Q: What happens to an off-topic answer? . . . 39
Q: How is the final percentage calculated? . . . 39
Q: Can the teacher change AI marks? . . . 39
Q: What is AI-teacher agreement? . . . 40
Q: Why background jobs? . . . 40
Q: How does local PDF generation work? . . . 40
Q: How is data integrity maintained? . . . 40
Q: What is the biggest technical limitation? . . . 40
Q: What security issue should be fixed first? . . . 40
Q: How would you prove accuracy? . . . 40
Q: What happens if Render sleeps? . . . 40
Q: What happens if local AI is off? . . . 40
Q: Why not trust the PDF text layer always? . . . 40

Q: What is human-in-the-loop AI? 40
Q: What did you personally implement/change? 40

26. Glossary **41**

27. Demonstration Checklist **42**

28. References and Source Basis **43**

1. Executive Summary

TwinEval AI is a deployed academic evaluation platform that connects paper creation, answer-key generation, answer-sheet processing, AI-assisted marking, teacher review, student feedback, analytics, and report export in one role-based web application.

The current design is deliberately hybrid. Account, paper, submission, and review data are centralized in a cloud API and PostgreSQL database. Computationally expensive and privacy-sensitive AI operations run on the user's own computer through a local Python service and Ollama. The browser therefore communicates with both the public API and localhost:8000.

Teachers create or upload a question paper, allow the local AI to split it into questions and generate structured answer keys, review the generated content, and publish the paper. Students choose a published paper, upload an answer sheet, and start a local OCR and evaluation job. Results are finalized in the API database. Teachers may review the submission, change the total or per-question marks, and add comments. Students then see the final score, question-wise feedback, teacher comments, analytics, and a PDF report.

The AI is decision support, not an autonomous authority. OCR errors, incomplete model answers, unusual correct phrasing, diagrams, formulae, and model variability can affect results. Teacher review is therefore part of the intended workflow, and the report explains where human validation is essential.

One-sentence explanation
TwinEval is a cloud-hosted assessment portal with a locally executed AI evaluator: the cloud stores identities and academic records, while each authorized user runs OCR and LLM processing on their own machine.

2. Problem Definition, Objectives, and Scope

2.1 Problem addressed

Manual evaluation of multi-question answer scripts is repetitive, slow, and difficult to scale. Students often receive only a total mark, while teachers must separately manage question papers, answer keys, marks, comments, and performance summaries.

A useful automation system must do more than compare exact words. It must ingest documents, locate questions and answers, recognize alternative phrasing, apply a marking rubric, explain deductions, preserve teacher authority, and maintain an auditable final result.

2.2 Core objectives

- Provide secure student and educator accounts using verified Gmail identities.
- Digitize question papers and answer scripts in PDF, image, DOCX, or TXT form.
- Generate editable question-wise model answers, keywords, concepts, expected points, diagrams, Bloom levels, and marking rubrics.
- Evaluate objective and descriptive answers with transparent question-level evidence and marks.
- Keep teachers in control through review, overrides, comments, and flags.
- Give students score summaries, detailed feedback, reviewed answer-script access, analytics, and downloadable reports.
- Support both local development and a deployed multi-user environment without placing local LLM workloads on the central server.

2.3 Scope boundaries

- The system assists academic evaluation; it does not replace institutional moderation or educator judgment.
- Local AI requires the Python service, Ollama, and the configured models to be installed on the computer performing the AI task.
- Handwriting and diagram recognition depend on scan quality and model capability.
- The current frontend intentionally uses local AI even though an optional hosted Gemini service remains in render.yaml as a legacy/alternative blueprint.

3. Project Evolution: Old Report vs Current System

Area	Earlier documented state	Current implemented state
Frontend	Served by the Node application	Static HTML/CSS/JS built to dist and deployed on Vercel
Backend	Local Express server	Express API deployed on Render; still runnable locally
Database	SQLite only	Supabase PostgreSQL in production; SQLite fallback for local use
File storage	Local uploads directory	Supabase Storage in production; local uploads fallback
Identity	Email/password and JWT	Verified Gmail password signup, Google Sign-In, JWT, profile/avatar persistence
AI topology	Node-to-local AI bridge	Deployed browser calls localhost AI directly; Node bridge retained for local/server paths
Models	gemma3:4b described as default	qwen3:4b-instruct for text generation/evaluation; gemma3:4b for vision/OCR
Teacher review	Described but partly unverified	Implemented overall and per-question overrides, comments, flags, reviewed status
Student feedback	Score and question breakdown	Reviewed script, teacher comments, question tabs, final marks, analytics, local PDF
Analytics	Generic or sample data	Live submission-derived student and class analytics using final reviewed marks
Deployment	Local/LAN emphasis	Vercel + Render + Supabase with local AI on each client

4. Users, Roles, and Use Cases

Role	Main responsibilities	Important permissions
Student	Create account, select paper, upload answer sheet, run local evaluation, view final feedback and analytics, export report	Can access only their own submissions through authenticated APIs
Teacher / Educator	Create papers, generate answer keys, publish papers, review student submissions, override marks, add comments, view class analytics	Paper ownership is checked for review and modification
Admin	Administrative role supported by authorization middleware	Can access operations that accept teacher or admin

4.1 Student use cases

- Register with Gmail and verify the email, or authenticate using Google Sign-In.
- Complete profile details such as roll number, department, semester, college, and avatar.
- View available published papers and upload an answer script up to 25 MB.
- Run OCR and evaluation using the local AI service and monitor job progress.
- Open the score summary, AI feedback workspace, teacher-reviewed script, analytics, and PDF report.
- Permanently delete the account through a confirmation-protected settings action.

4.2 Teacher use cases

- Create a paper from typed questions or uploaded documents.
- Generate, inspect, and finalize answer keys before publishing.
- View evaluated submissions and distinguish pending review from reviewed work.
- Compare AI marks with the answer text, adjust marks, add teacher comments, and flag questions.
- Analyze final class performance, AI-teacher agreement, grade distribution, hardest questions, and Bloom-level averages.
- Export class analytics as CSV.

5. Current System Architecture

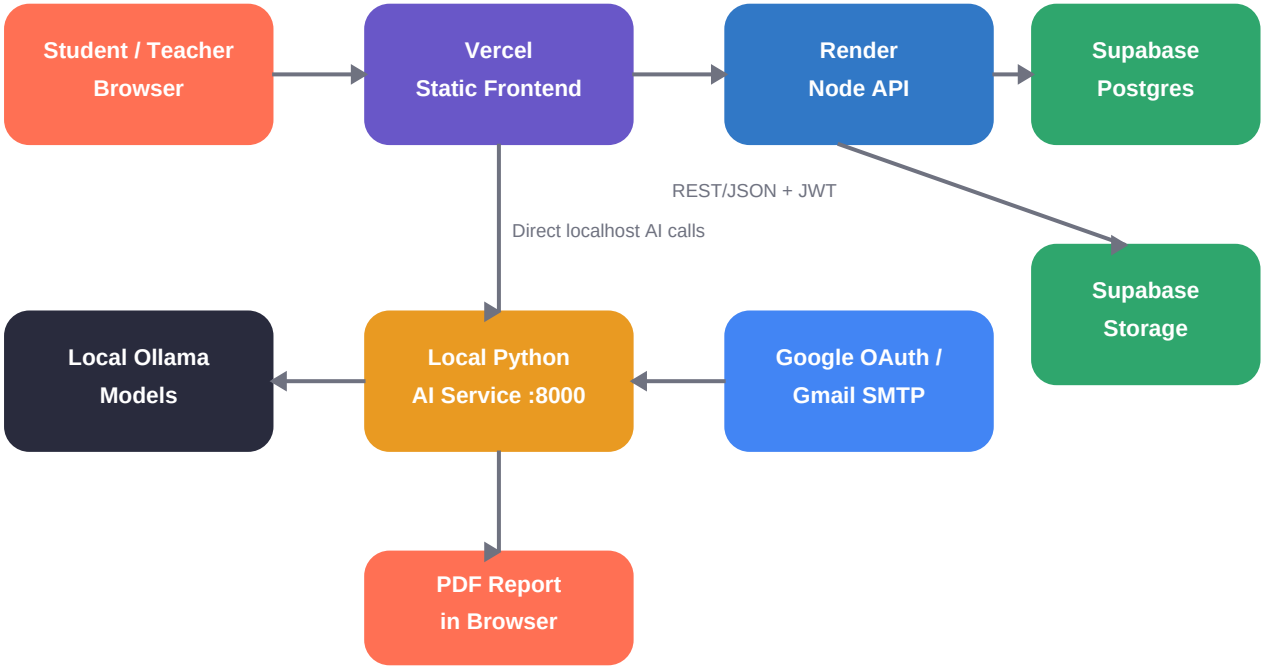


Figure 5.1 - Current hybrid cloud/local architecture.

5.1 Why the architecture is hybrid

A centralized API is necessary for multiple accounts to share papers, submissions, reviews, and analytics. PostgreSQL provides persistent relational data and Supabase Storage preserves uploaded documents and avatars across deployments.

AI processing is separated because free cloud web instances are unsuitable for multi-gigabyte local models and sustained OCR workloads. The browser calls the local AI service directly at 127.0.0.1:8000. This keeps model execution and raw document inference on the user's computer, but it also means every computer that performs AI work must run the local service.

The Vercel build injects the Render API base URL and fixes the AI URL to localhost. This prevents a cloud environment variable from silently routing private documents to a hosted AI provider.

5.2 Component responsibilities

Component	Technology	Responsibility
Browser UI	HTML5, CSS3, vanilla JavaScript	Role-based screens, validation, API calls, local AI calls, charts, downloads
Frontend hosting	Vercel	Builds public/ into dist/ and serves static assets globally
Central API	Node.js 22+, Express	Identity, authorization, papers, submissions, finalization, review, analytics data
API hosting	Render	Runs npm start and exposes /health and REST routes
Database	Supabase PostgreSQL	Persistent relational records and foreign-key relationships
Object storage	Supabase Storage	Persistent avatars and uploaded academic documents
Local AI service	Python, Flask, Flask-CORS	Extraction, OCR, question splitting, answer keys, evaluation, PDF generation
Local model runtime	Ollama	Runs qwen3:4b-instruct and gemma3:4b
External identity	Google Identity Services	Verifies Google ID tokens and links Gmail identity
Email	SMTP / Gmail App Password	Sends 24-hour verification links for password signup

6. Technology Stack and Why Each Technology Is Used

Layer	Technology	Reason for selection
Frontend	HTML, CSS, vanilla JavaScript	Lightweight static deployment, no framework build dependency, direct control over role-based pages
API	Node.js and Express	Asynchronous I/O, simple REST routing, broad ecosystem, shared JavaScript language with frontend
Security	Helmet, CORS, JWT, bcryptjs	Security headers, cross-origin access, stateless sessions, password hashing
Validation	express-validator	Server-side checks for Gmail, passwords, roles, scores, and student data
Uploads	Multer memory storage	Validates file type/size and forwards buffers to local or Supabase storage
Production DB	PostgreSQL via pg	Concurrent multi-user cloud persistence with relational integrity
Local DB	node:sqlite	Zero-install local fallback using the same logical schema
Storage	Supabase JS client	Server-side object upload and persistent public URLs
Email	Nodemailer	SMTP verification email delivery
Google auth	google-auth-library	Back-end verification of Google ID tokens and audience
AI API	Python Flask + Flask-CORS	Small local HTTP service callable from browser JavaScript
Document parsing	PyMuPDF, python-docx, Pillow	PDF text/images, DOCX parsing, image handling
Text AI	qwen3:4b-instruct	Local question splitting, answer-key generation, grading explanations
Vision AI	gemma3:4b	OCR and transcription of scanned or handwritten pages
Semantic analysis	sentence-transformers, scikit-learn	Meaning-aware comparison and concept similarity
Image processing	OpenCV, NumPy	Preprocessing and image-related support
Reports	ReportLab	Deterministic PDF creation from evaluation data
Deployment	Vercel, Render, Supabase	Static frontend, managed API, managed database and object storage

7. End-to-End Functional Workflow

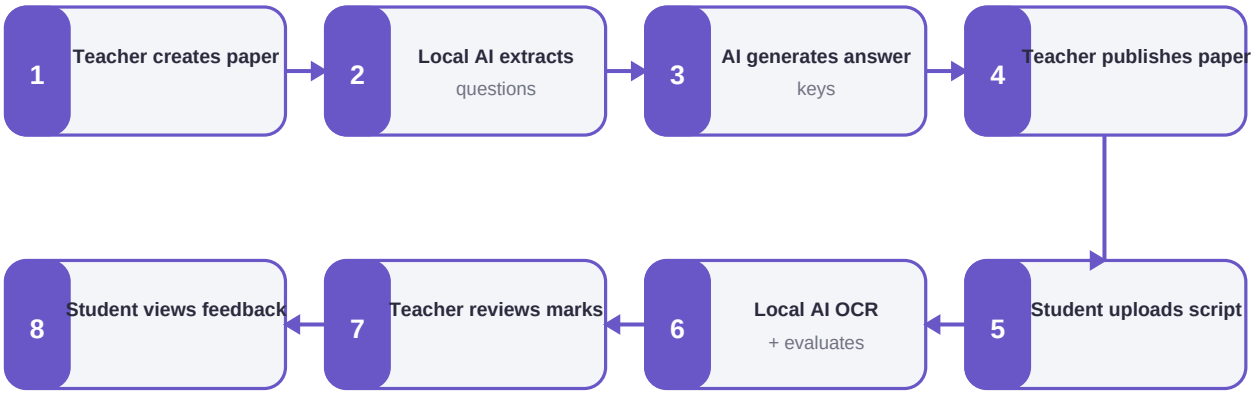


Figure 7.1 - Complete paper-to-feedback workflow.

7.1 Question paper workflow

- The teacher enters title, subject, topic, default marks, and pass percentage.
- Questions may be typed or uploaded as PDF, DOCX, TXT, PNG, JPG, or JPEG.
- The browser sends the file directly to the local AI job endpoint /jobs/generate-paper.
- The AI extracts text, identifies question boundaries and marks, handles grouped instructions such as 'answer any N', and generates a structured answer key for each question.
- The teacher reviews the generated questions and answer keys. Finalization stores QuestionPaper, Question, and AnswerKey rows in the central API database.
- Publishing changes the paper status from draft to published so students can select it.

7.2 Student evaluation workflow

- The student selects a published paper and uploads an answer script.
- The central API creates a processing submission and stores the uploaded document persistently.
- The browser sends the document and paper definition to the local /jobs/evaluate-submission endpoint.
- The AI extracts text, divides it into question-wise answers, evaluates each response, and calculates totals, percentage, grade, pass/fail, strengths, weaknesses, and review flags.
- The browser finalizes the job by sending the evaluation result to the central API, which writes Submission and QuestionResult records.
- The teacher reviews the outcome; the student sees final reviewed data after the status becomes reviewed.

8. Authentication, Account Lifecycle, and Profiles

8.1 Password-based Gmail registration

Registration accepts only syntactically valid addresses ending in @gmail.com. The email is normalized, the password must contain at least six characters, and bcrypt hashes the password before storage. Student registration also requires roll number, department, and semester so a linked Student row can be created.

If SMTP is configured, the server creates a cryptographically random verification token, stores only its hash and expiration, and sends a verification link that expires after 24 hours. Login is rejected until verification is complete.

8.2 Continue with Google

Google Identity Services returns an ID credential to the browser. The credential is sent to the Node API, where google-auth-library verifies the signature, audience, email_verified claim, Gmail domain, and stable Google subject identifier.

If the selected Gmail already belongs to a TwinEval account, the server links the Google subject when necessary and returns a JWT session. If no account exists, the browser opens the create-account page with the verified name, email, avatar, and Google credential prefilled. The user still selects the role and completes academic profile fields. Google signup does not require a second SMTP verification because Google has already verified ownership of the address.

8.3 Session and profile behavior

- JWT payload contains user_id, email, name, role, and student_id where applicable.
- Default token lifetime is seven days unless JWT_EXPIRES_IN is changed.
- The browser stores the token and cached user profile in localStorage and sends Authorization: Bearer .
- Profile updates are synchronized with the server. Avatars are uploaded to persistent storage and reloaded after login.
- Settings include permanent account deletion with password/confirmation safeguards. Database cascades remove associated owned records where configured.

9. AI Engine and Evaluation Intelligence

9.1 Local service design

The AI engine is a Flask service with CORS enabled so the deployed Vercel page can call localhost. Long operations run as background jobs. The UI starts a job, receives a job identifier, and polls /jobs/ for stage, progress, result, or error. This avoids fragile single HTTP requests during multi-page OCR and per-question inference.

All Ollama calls pass through a centralized timeout-aware wrapper. The default chat timeout is 90 seconds with retry support. This prevents one stalled model request from freezing the entire evaluation indefinitely and allows fallback logic to continue or return an actionable error.

9.2 Document extraction

- TXT: decoded directly as UTF-8 with error tolerance.
- DOCX: paragraph and table text extracted with python-docx.
- Digital PDF: PyMuPDF extracts the embedded text layer page by page.
- Scanned PDF: image coverage and garbled-text heuristics reject unreliable embedded OCR; the page is rendered at 300 DPI and sent to the vision model.
- PNG/JPG/JPEG: saved temporarily and transcribed by gemma3:4b through Ollama.
- Optional fallbacks: Gemini OCR if AI_PROVIDER=gemini and a key exists; PaddleOCR only if installed.

9.3 Question and answer segmentation

QuestionAgent detects numbered questions, extracts explicit marks, filters headers/footers, and recognizes group instructions. It uses deterministic parsing first and LLM assistance when needed. StudentAnswerAgent similarly maps extracted answer text to question numbers with regex-first and LLM fallback behavior.

The answer-key generator creates a structured object containing model_answer, keywords, concepts, expected_points, advantages/disadvantages, applications, algorithms/steps, formulae, diagrams, Bloom level, and a marks-distribution rubric. This object is saved as JSON in AnswerKey.data.

9.4 Answer evaluation

Answer type / condition	Evaluation behavior
Objective / 0.5-1 mark	Direct match of normalized option letter, true/false, phrase, or strong keyword overlap; normally full or zero marks
Descriptive	Rubric-aware grading based on expected concepts, keywords, points, technical correctness, logical flow, applications, algorithms, formulae, diagrams, and completeness
Empty	Zero marks with explicit no-answer feedback
Clearly off-topic	Zero marks, improvement guidance, and needs_review flag for human checking
Alternative phrasing	Stem-aware term matching and semantic comparison reduce dependence on exact wording
Diagram requirement	Textual references and expected labels are used as a conservative proxy; true visual diagram assessment remains limited

9.5 Result object

```
{
  marks, max_marks, percentage, grade,
  marks_breakdown, found_concepts, missing_concepts,
  found_keywords, missing_keywords, missing_points,
  missing_diagrams, strengths, improvements, suggestions,
  confidence, bloom_level, topic_relevance,
  feedback, evidence_map, needs_review, review_reason
}
```

Human-in-the-loop rule

Generated answer keys must be checked before publishing, and AI-evaluated marks must remain reviewable. A confident model output can still be wrong if OCR, the model answer, or the rubric is wrong.

10. Scoring, Grades, Review, and Final Marks

10.1 Grade scale

Percentage	Grade
>= 90	A+
80-89.99	A
70-79.99	B+
60-69.99	B
50-59.99	C
40-49.99	D
< 40	F

10.2 Overall calculation

For a submission with awarded question marks m_i and maximum question marks M_i:

Formula
Total marks = sum(m_i). Maximum marks = sum(M_i). Percentage = (Total marks / Maximum marks) x 100. Pass/fail is compared with the paper's configured pass percentage.

10.3 Teacher review

The review workspace shows extracted answers, AI marks, maximum marks, and AI feedback. The teacher can save an overall override, add comments, override individual question results through the API, or flag a question for re-review. Saving the review changes the submission status to reviewed.

Student pages prefer teacher_override_marks whenever it exists. This guarantees the score summary, feedback workspace, analytics, and exported final score represent teacher-approved data rather than the original AI suggestion.

10.4 Analytics calculations

Metric	Current calculation
Student average	Average of final percentage for evaluated/reviewed submissions
Cumulative GPA	Final average percentage divided by 10, capped at 10
AI evaluator accuracy	For reviewed items: 100 - |teacher marks - AI marks| / maximum marks x 100
Class mean	Mean of final teacher-reviewed percentages
AI-teacher agreement	Average score similarity between AI total and teacher-approved total
Outstanding score count	Number of final percentages >= 90
Grade distribution	Student count grouped by grade computed from final percentage
Hardest question	Question with the lowest average final question percentage across responses
Bloom radar	Average final question percentage by stored Bloom level

11. Database Design and Data Model

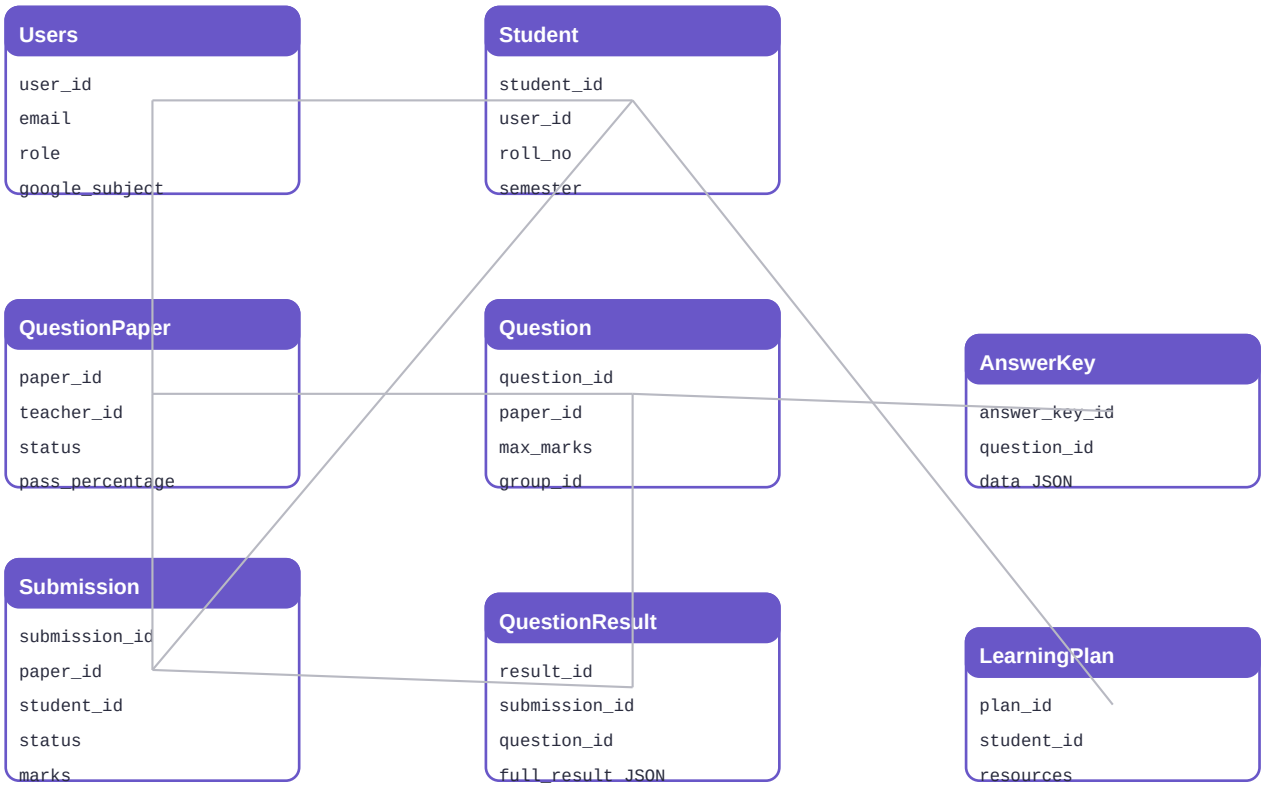


Figure 11.1 - Main operational data relationships. Three legacy analytics tables are omitted from the drawing for clarity.

11.1 Dual database adapter

The same model layer supports two database modes. When DATABASE_URL exists, pg connects to PostgreSQL with production SSL. Otherwise Node's built-in node:sqlite opens the local database, enables foreign keys, WAL mode, and a busy timeout. SQL placeholders and insert return values are adapted so existing models work in both modes.

Schema initialization runs automatically at startup. PostgreSQL converts SQLite AUTOINCREMENT definitions to identity columns and creates updated_at triggers. SQLite performs additive migrations for avatar, academic profile, email verification, and Google subject fields.

11.2 Table dictionary

Table	Purpose	Key relationships
Users	Login identity, role, academic profile, avatar, verification, Google identity	Parent of Student and teacher-owned QuestionPaper
Student	Student academic identity and roll/semester data	Links Users to Submission, EvaluationResult, LearningPlan
QuestionPaper	Paper metadata, pass percentage, source file, draft/published status	Owned by Users; parent of Question and Submission
Question	Ordered question text, marks, topic, AI ID, optional group rules	Belongs to QuestionPaper; parent of AnswerKey and QuestionResult
AnswerKey	Structured generated answer-key JSON	One-to-one with Question
Submission	Uploaded script, extracted text, AI totals, final override, status, comments	Belongs to QuestionPaper and Student
QuestionResult	Per-question answer, marks, feedback JSON, flags, override	Belongs to Submission and Question
EvaluationResult	Legacy/general performance evaluation record	Belongs to Student
LearningPlan	Weak topics, resources, practice tasks, schedule	Belongs to Student
TeacherAnalytics	Legacy persisted department analytics	Independent summary table

11.3 Important states

Object	States
QuestionPaper.status	draft, published
Submission.status	processing, evaluated, reviewed, failed
Submission.pass_fail	Pass, Fail, or null while processing
Users.role	admin, teacher, student

12. REST API Design

All API responses use a standard envelope: success, message, and data for success; success=false and message for errors. Authenticated routes expect a Bearer JWT.

Method and route	Access	Purpose
POST /auth/register	Public	Create verified Gmail account; supports Google-assisted signup
POST /auth/login	Public	Password login after email verification
GET /auth/google/config	Public	Return configured Google client ID
POST /auth/google	Public	Verify Google token; login or return signup prefill
GET /auth/verify-email	Public	Consume verification token
GET /auth/me	JWT	Return current profile
PUT /auth/profile	JWT	Update profile
POST /auth/avatar	JWT + upload	Persist profile image
DELETE /auth/account	JWT	Permanently remove account
POST /papers/generate-job	Teacher/admin	Server-bridge generation path
GET /papers/generate-job/:jobId	Teacher/admin	Poll generation job
POST /papers/finalize	Teacher/admin	Store generated paper/questions/keys
GET /papers	Current route public	List/filter papers; frontend uses published or owned views
PATCH /papers/:id/publish	Teacher/admin	Publish a draft
DELETE /papers/:id	Teacher/admin	Delete owned paper
POST /submissions	Student + upload	Create processing submission and persist file
POST /submissions/:id/finalize	Student	Store AI result
POST /submissions/:id/mark-failed	Student	Record processing failure
GET /submissions	JWT	Role-filtered submission list
GET /submissions/:id	JWT	Submission with question results
GET /submissions/:id/report	JWT	Report path with local-generation fallback in UI
PUT /submissions/:id/override	Teacher/admin	Save final total and comments
PUT /submissions/:id/questions/:resultId/override	Teacher/admin	Save final question mark
POST /submissions/:id/questions/:resultId/flag	JWT	Flag result for review

Security note
Some legacy GET routes in students, evaluations, plans, analytics, dashboard, and papers remain public in the route files. Production hardening should add authentication and ownership/role checks to every endpoint that exposes academic data.

13. Frontend Application

13.1 Page map

Page	Role / purpose
index.html	Sign in with password or Google; select student/teacher portal
signup.html	Student or educator account creation; Google-prefilled signup supported
verify-email.html	Completes email verification
student-dashboard.html	Scores, recent activity, paper availability, progress
upload-answer.html	Paper selection and answer script upload
processing.html	Evaluation progress
results.html	Final score summary and PDF report
feedback.html	Question-wise AI feedback, teacher comments, submitted script
analytics.html	Student score trend, Bloom mastery, rubric and AI accuracy
teacher-dashboard.html	Paper/submission overview and pending review
teacher-upload.html	Paper creation and local answer-key generation
teacher-review.html	AI result inspection and teacher approval/override
teacher-analytics.html	Final-score class analytics and CSV export
profile.html	Shared profile and persistent avatar
settings.html	Theme/preferences and permanent account deletion

13.2 Shared client behavior

- script.js provides API base resolution, JWT fetch wrapper, localStorage session cache, user/avatar rendering, dropdowns, toasts, and SVG charts.
- redesign.js normalizes role-specific navigation, injects AI Feedback into every student page, sets role labels, and loads notifications.
- ai-local.js isolates direct localhost AI requests and returns clear startup errors when the service is unavailable.
- Role-specific modules load live data rather than relying on hardcoded sample values.

14. File Handling, Storage, and Report Generation

14.1 Upload policy

Multer uses memory storage, limits each upload to 25 MB, and permits .pdf, .png, .jpg, .jpeg, .docx, and .txt. The server sanitizes filenames before persistence.

If Supabase credentials are configured, buffers are uploaded to the twineval-uploads bucket and a public URL is stored. Otherwise the file is written to the local uploads directory and exposed through /uploads.

14.2 Evaluation report PDF

The results page constructs a report payload from the final submission, question text, answer keys, question results, and teacher overrides. It asks the local AI service to generate the PDF with ReportLab and downloads the returned Blob in the browser.

This local path solves the earlier Render 502 problem caused by depending on a remote AI/PDF service. It also ensures teacher-reviewed marks are reflected in the downloaded document.

14.3 CSV analytics

Student analytics exports paper, subject, date, status, final marks, maximum marks, and final percentage. Teacher analytics exports student, roll number, paper, status, final marks, final percentage, and final grade.

15. Current Application Screens

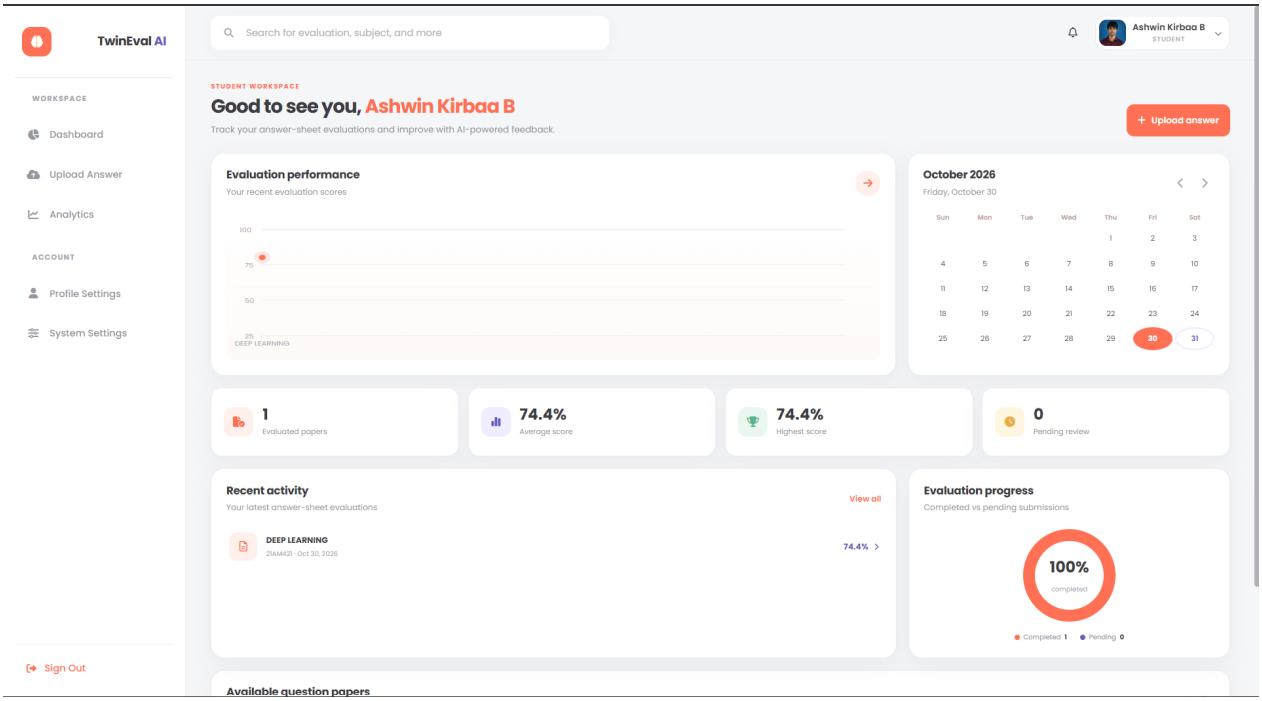


Figure 15.1 - Student dashboard with live evaluation summary and recent activity.

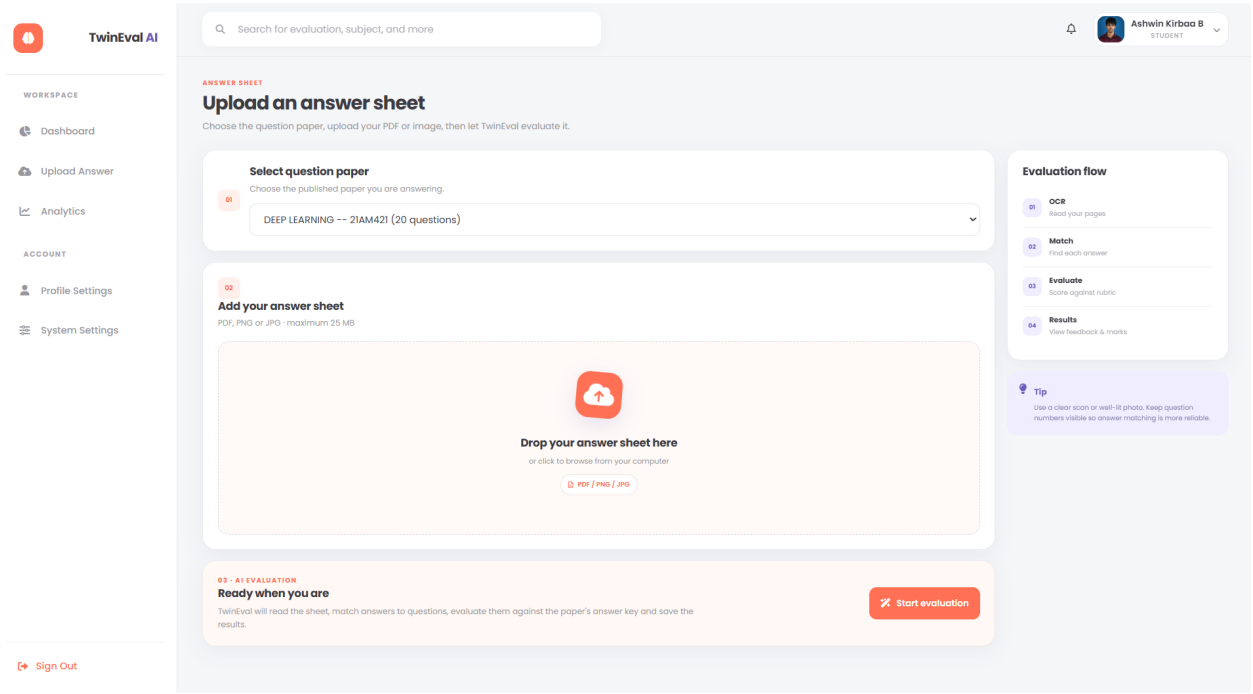


Figure 15.2 - Student answer-sheet upload workflow and local AI evaluation entry point.

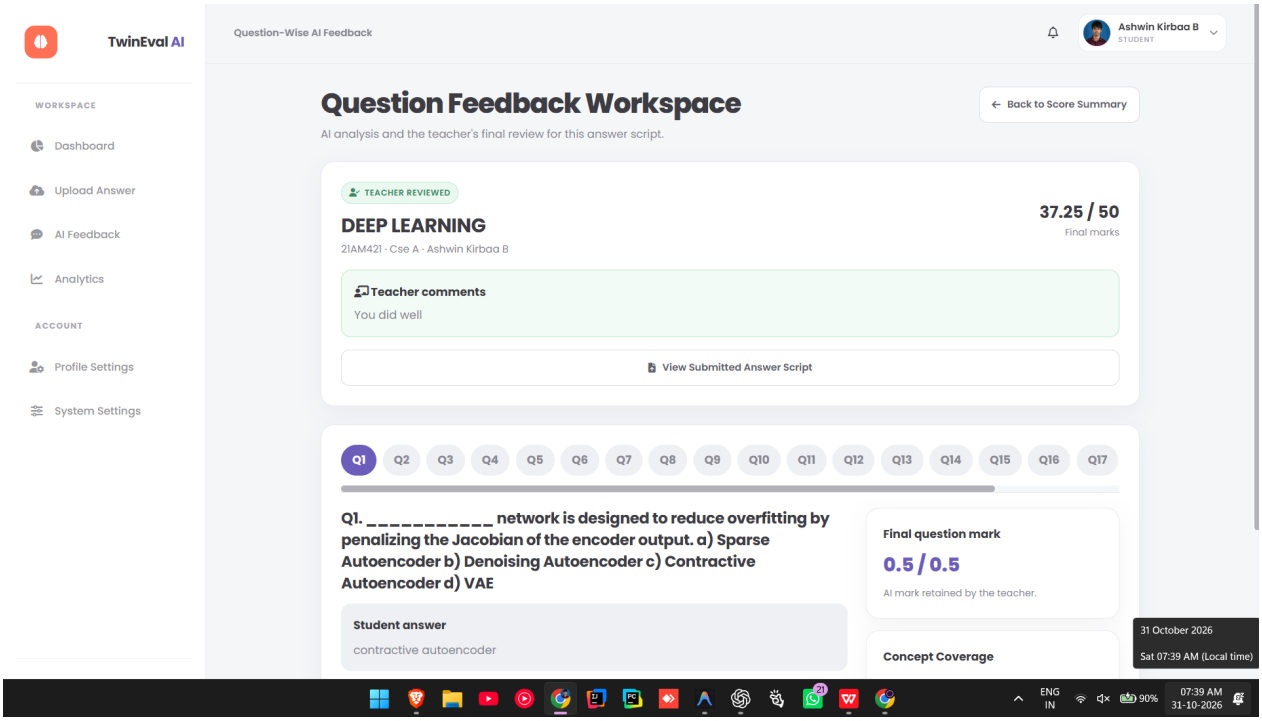


Figure 15.3 - Question-wise AI feedback with teacher-reviewed marks and comments.

16. Deployment Architecture and Configuration

16.1 Production deployment

Service	Configuration
Vercel	Build: npm run build:frontend; output: dist; TWINEVAL_API_BASE points to Render API
Render API	Runtime: Node; build: npm ci; start: npm start; health: /health
Supabase	PostgreSQL connection through DATABASE_URL; Storage through project URL and service-role key
Local AI	Run start-ai-windows.bat on every AI-using computer; health at http://127.0.0.1:8000/health
Google Cloud	OAuth web client must authorize exact Vercel and localhost JavaScript origins
Gmail SMTP	Sender Gmail with two-step verification and a 16-character App Password

16.2 Important environment variables

Variable	Used by	Meaning
DATABASE_URL	API	Production PostgreSQL connection string
DB_PATH	API	Local SQLite path when DATABASE_URL is absent
JWT_SECRET	API	Signs and verifies sessions; must be long and private
JWT_EXPIRES_IN	API	Token lifetime, default 7d
PUBLIC_APP_URL	API/email	Frontend origin used in verification links
GOOGLE_CLIENT_ID	API/frontend	Google OAuth web client audience
SMTP_HOST/PORT/SECURE	API	SMTP transport
SMTP_USER/PASS/FROM	API	Verification sender credentials
SUPABASE_URL	API	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	API only	Privileged storage access; never expose to browser
SUPABASE_STORAGE_BUCKET	API	Default twineval-uploads
TWINEVAL_API_BASE	Vercel build	Public API origin
AI_TEXT_MODEL	Local AI	Default qwen3:4b-instruct
AI_VISION_MODEL	Local AI	Default gemma3:4b
OLLAMA_TIMEOUT_SECONDS	Local AI	Per-call timeout, default 90
OLLAMA_ANSWER_KEY_TIMEOUT_SECONDS	Local AI	Longer answer-key timeout, configured as 240

16.3 Why secrets must stay server-side

Database passwords, JWT_SECRET, SMTP App Password, Supabase service-role key, and any Gemini key must never be committed or placed in Vercel browser variables. Browser-delivered JavaScript is public. Only non-secret origins such as TWINEVAL_API_BASE belong in the frontend build.

17. Installation and Execution Guide

17.1 Local prerequisites

- Node.js 22.5 or newer and npm.
- Python environment compatible with the listed AI dependencies.
- Ollama desktop/runtime.
- qwen3:4b-instruct and gemma3:4b pulled into Ollama.
- A configured .env for API/database/auth/email settings.

17.2 Windows startup

```
1. Run start-ai-windows.bat
2. Confirm http://127.0.0.1:8000/health
3. For local web development, run start-windows.bat
4. Open http://localhost:5000

For the deployed site, only steps 1 and 2 are needed locally,
then open https://twineval-ai.vercel.app
```

17.3 Manual commands

```
npm install
npm run setup
npm start

python -m venv .venv
.venv\Scripts\activate
pip install -r ai-engine/requirements.txt
python ai-engine/ai_server.py
```

18. Security and Privacy Assessment

18.1 Implemented controls

- bcrypt password hashing; plaintext passwords are not stored.
- JWT verification and role middleware on sensitive write operations.
- Google ID tokens are verified server-side against the configured audience.
- Email verification tokens are random, hashed in storage, and time limited.
- Teacher ownership checks protect paper and submission review operations.
- Upload extension and 25 MB size validation.
- Helmet security headers with same-origin-allow-popups for Google Sign-In.
- Parameterized database queries through adapters.
- Service-role storage credential remains on the server.

18.2 Current risks and recommended hardening

Risk	Why it matters	Recommended action
Open CORS	Any origin can call public API routes	Restrict CORS to deployed and local origins
Fallback JWT secret	A known fallback weakens misconfigured deployments	Refuse startup in production when JWT_SECRET is missing
Public legacy GET routes	Could expose student or analytics data	Require JWT and apply role/ownership filters
JWT in localStorage	XSS can steal tokens	Strengthen CSP; consider secure HttpOnly cookies
Public storage bucket	URLs may be accessible if known	Use private bucket and signed URLs for answer scripts
No rate limiting shown	Login/upload abuse can consume resources	Add per-IP and per-account rate limits
Account deletion impact	Cascades may permanently remove academic records	Add retention policy, export, audit, and institutional approval
Local AI CORS	Local service is reachable from browser origins	Restrict allowed origins and validate job payloads

19. Testing and Validation Strategy

19.1 Functional test matrix

ID	Scenario	Expected result
T01	Password signup with valid Gmail and SMTP	Verification email sent; login blocked until verified
T02	Google Sign-In for existing account	Immediate JWT session and correct dashboard
T03	Google Sign-In for new account	Signup prefilled with verified name/email/avatar
T04	Student attempts teacher paper write route	HTTP 403
T05	Upload unsupported extension or >25 MB	Clear validation error
T06	Generate paper from typed PDF	Questions, marks, keys, and rubrics appear for review
T07	Generate paper from scanned PDF	Per-page OCR progress and extracted questions
T08	Evaluate objective answers	Exact correct answers receive full marks; wrong answers zero
T09	Evaluate descriptive paraphrase	Semantically correct alternative wording receives appropriate credit
T10	Off-topic answer	Zero/low mark with needs_review flag
T11	Teacher override	Student score, feedback, analytics, CSV, and PDF use final marks
T12	Render cold start	Frontend reports delay gracefully; request succeeds after wake-up
T13	Local AI not running	Actionable error instructs user to start local engine
T14	PDF report	Downloads and opens with correct totals and question data
T15	Account deletion	Confirmation required; account and permitted dependent data removed

19.2 Evaluation quality testing

A credible accuracy claim requires a benchmark set graded independently by qualified educators. For every answer, retain the question, approved answer key, extracted student text, AI mark, human mark, and reason for disagreement.

Recommended measurements include mean absolute mark error, exact agreement for objective questions, weighted kappa for grade bands, correlation, false off-topic rate, OCR word error rate, average latency per page/question, and the percentage of answers sent to human review. Results should be reported by question type and mark range, not only as one overall percentage.

20. Current Results and Demonstrated Capabilities

- Frontend and API are deployed separately and configured to communicate across origins.
- Supabase PostgreSQL preserves multi-user accounts and academic records; Storage preserves avatars and documents.
- Google Sign-In supports existing-account login and new-account prefill.
- Password signup supports real Gmail verification through SMTP.
- Question-paper generation and answer-sheet evaluation use local background AI jobs.
- Teachers can review and approve AI output; students can see reviewed comments and final question marks.
- Student AI Feedback is present in the navigation across all student pages.
- Student and teacher analytics derive values from live submissions, and teacher analytics use final reviewed marks.
- PDF reports are generated by the local AI service; CSV analytics are generated in the browser.
- Profile avatars are stored persistently and restored after login.
- Permanent account deletion is available in settings.

Do not overclaim
The repository and deployed UI demonstrate system capability, but they do not by themselves prove grading fairness or accuracy. Such claims require the benchmark process in Section 19.2.

21. Limitations and Engineering Trade-offs

Limitation	Effect
Local AI setup required	Users cannot generate/evaluate/report until Ollama, models, and Python service are running
Large local models	Initial model download is several GB; CPU-only inference can be slow
OCR variability	Blur, handwriting, formulas, and page layout can change extracted text and downstream marks
Generated answer-key quality	An incorrect key can systematically affect an entire class unless reviewed
Text proxy for diagrams	The evaluator cannot reliably judge visual correctness of a hand-drawn diagram from text alone
Render free-tier sleep	First API request after inactivity may take around a minute
Static multi-page frontend	Shared behavior is duplicated or injected rather than componentized
Legacy modules/routes	EvaluationResult/LearningPlan/TeacherAnalytics and some public endpoints coexist with the newer paper/submission flow
No formal audit log	Mark and account changes are not preserved as a complete immutable history

22. Future Enhancements

- Add authenticated, ownership-aware access to every academic read endpoint.
- Replace public document URLs with private storage and short-lived signed URLs.
- Add immutable audit records for answer-key edits, teacher overrides, account deletion, and publication.
- Provide an OCR correction screen before evaluation.
- Use real multimodal diagram assessment with explicit teacher confirmation.
- Add queue-based asynchronous processing and resumable jobs for large classes.
- Provide institutional SSO and domain allowlists in addition to Gmail.
- Create rubric versioning so every submission records the exact key/rubric used.
- Build calibration dashboards comparing AI and teacher marks by subject and question type.
- Add automated unit, integration, security, accessibility, and end-to-end tests in CI.
- Add backup/restore procedures and retention controls for Supabase data.
- Convert the multi-page frontend into reusable components if maintenance complexity grows.

23. Current Repository Structure

```
TwinEval_AI/
|-- public/                 Static frontend pages, CSS, and browser JS
|-- src/
|   |-- config/             SQLite/PostgreSQL database adapter
|   |-- controllers/        Auth, paper, submission, analytics logic
|   |-- middleware/         JWT, roles, validation, uploads, errors
|   |-- models/             SQL data-access layer
|   |-- routes/             Express route definitions
|   `-- services/           AI bridge, email, storage, scoring helpers
|-- ai-engine/
|   |-- agents/             Question, key, OCR, grading, Bloom, feedback
|   |-- services/           End-to-end paper evaluation pipeline
|   |-- utils/              File readers, Ollama client, PDF generator
|   |-- dataset/            Concepts and calibration exemplars
|   `-- ai_server.py        Flask service and background jobs
|-- docs/                   API, deployment, network, schema documents
|-- scripts/                Setup, import, and Vercel frontend build
|-- render.yaml             Render Blueprint
|-- vercel.json             Vercel build configuration
|-- server.js               Node API entry point
|-- start-*.bat/.sh         Local startup scripts
`-- package.json            Node dependencies and scripts
```

24. Troubleshooting Guide

Symptom	Likely cause	Resolution
Local AI unavailable	Python service or Ollama not running	Run start-ai-windows.bat; open /health; verify Ollama app
Model not found	Model not pulled or model name differs	ollama pull qwen3:4b-instruct and ollama pull gemma3:4b
Generation freezes	Model overloaded or old unbounded call	Use current timeout wrapper; inspect Ollama; retry after model loads
Google origin_mismatch	Exact frontend origin missing	Add https://twineval-ai.vercel.app to Authorized JavaScript origins
Google popup blank/frozen	COOP or browser privacy behavior	Use current same-origin-allow-popups header; allow Google popup/cookies
Verification email fails	SMTP password/host/from invalid	Use Gmail App Password; verify Render environment variables
Registration HTTP 500	Database migration or SMTP/storage error	Inspect Render logs; confirm schema initialized and credentials valid
Database password authentication failed	Incorrect/uncoded Supabase connection string	Reset password and URL-encode special characters
PDF download 502	Old server-side hosted AI path	Run local AI and use current browser local PDF generation
Profile image missing	Old local-only path or storage misconfiguration	Configure Supabase Storage; re-upload avatar
Render service slow first time	Free instance cold start	Wait for wake-up or use paid always-on plan
Analytics differs from final score	Old build used AI percentage	Deploy commit 6e587f4; refresh cache; final teacher marks are now used

25. Viva and Demonstration Question Bank

Q: What is TwinEval AI?

A: A deployed academic evaluation platform that automates document extraction and produces rubric-based marking suggestions, while preserving teacher review and giving students detailed feedback and analytics.

Q: What is the main innovation?

A: The integration of question-paper generation, local OCR/LLM evaluation, teacher approval, student feedback, and cloud persistence in one hybrid system.

Q: Why is it called hybrid?

A: The frontend, API, database, and storage are cloud-hosted, but AI inference runs locally on each user's computer.

Q: Why not run Ollama on Render?

A: Local models require more memory, storage, and compute than a free web instance can reliably provide. Local execution also reduces external exposure of exam content.

Q: Why Vercel?

A: The frontend is static and Vercel provides simple Git-based deployment, global asset delivery, and build-time environment injection.

Q: Why Render?

A: It provides a managed Node runtime, environment variables, health checks, and Git-based deployment for the Express API.

Q: Why Supabase?

A: It supplies managed PostgreSQL for persistent relational data and Storage for files that must survive Render restarts.

Q: Why keep SQLite support?

A: It makes local development portable and requires no external database server while sharing the same logical model layer.

Q: What is JWT?

A: A signed token that carries user identity and role. The API verifies it on protected requests instead of storing a server-side session.

Q: How are passwords protected?

A: bcrypt hashes passwords with a salt; login compares the supplied password against the stored hash.

Q: How does Google Sign-In work?

A: The browser receives a Google ID token, the backend verifies its signature/audience/verified Gmail, then logs in an existing user or returns verified signup data.

Q: Why is SMTP still needed?

A: Password-based signup does not have Google's proof of ownership, so the server emails a time-limited verification link.

Q: How are uploads protected?

A: The API restricts extensions and enforces a 25 MB limit. Production files are uploaded server-side to Supabase Storage.

Q: How does PDF OCR work?

A: PyMuPDF uses embedded text when trustworthy. Scanned or garbled pages are rendered at 300 DPI and transcribed by the local vision model.

Q: Why use two models?

A: qwen3:4b-instruct is optimized for text generation and reasoning, while gemma3:4b supports image/vision input for OCR.

Q: How are questions detected?

A: Regex and heuristics detect numbering, marks, metadata, and grouped instructions; an LLM fallback handles irregular formats.

Q: How are objective answers graded?

A: Normalized answer matching checks option letters, true/false forms, exact phrases, and strong keyword overlap; short objective items receive full or zero marks.

Q: How are descriptive answers graded?

A: The evaluator checks rubric criteria, expected concepts, keywords, points, technical correctness, completeness, flow, applications, algorithms, formulae, and diagram evidence.

Q: What is semantic similarity?

A: It compares meaning rather than exact wording, helping recognize valid paraphrases. It is evidence, not proof of factual correctness.

Q: What is Bloom's taxonomy used for?

A: Each question/result is classified into cognitive levels such as Remember, Understand, Apply, Analyze, Evaluate, and Create for skill analytics.

Q: What happens to an off-topic answer?

A: It receives zero or very low marks, explanatory feedback, and a needs_review flag so a teacher can check the decision.

Q: How is the final percentage calculated?

A: The sum of awarded question marks is divided by the sum of maximum marks and multiplied by 100.

Q: Can the teacher change AI marks?

A: Yes. The teacher can save an overall final score, comments, and per-question overrides. Student views prefer the override.

Q: What is AI-teacher agreement?

A: It is 100 minus the normalized absolute difference between AI marks and teacher-approved marks, averaged across reviewed submissions.

Q: Why background jobs?

A: OCR plus one model call per question can exceed normal request times. Jobs allow progress polling, retries, and clear error states.

Q: How does local PDF generation work?

A: The results page sends final evaluation data to localhost; ReportLab produces the PDF; the browser downloads the Blob.

Q: How is data integrity maintained?

A: Foreign keys relate users, students, papers, questions, keys, submissions, and results; cascades and unique constraints prevent orphaned/duplicate records.

Q: What is the biggest technical limitation?

A: Evaluation quality depends on OCR, the approved answer key, model behavior, and hardware. Teacher review and benchmark testing remain essential.

Q: What security issue should be fixed first?

A: Protect every academic read endpoint and remove the production fallback JWT secret; then restrict CORS and storage access.

Q: How would you prove accuracy?

A: Use a representative educator-graded benchmark and report mark error, objective accuracy, grade agreement, OCR error, false off-topic rate, and latency by question type.

Q: What happens if Render sleeps?

A: The first API request may be delayed while the free instance wakes. The local AI is independent of that cold start.

Q: What happens if local AI is off?

A: Non-AI account/data pages still use the cloud API, but generation/evaluation/report actions return a clear instruction to start the local engine.

Q: Why not trust the PDF text layer always?

A: Scan applications may embed low-quality OCR. Image-coverage and language heuristics detect suspicious pages and force real OCR.

Q: What is human-in-the-loop AI?

A: AI proposes structured outputs, but a human validates the key and final marks, especially for ambiguous or high-impact decisions.

Q: What did you personally implement/change?

A: The current repository adds deployment, Supabase persistence, Google and email authentication, local AI routing, answer generation/evaluation reliability, review/feedback, PDF fixes, avatars, account deletion, and live analytics corrections.

26. Glossary

Term	Meaning in TwinEval
API	HTTP interface used by browser, Node server, and local AI service
CORS	Browser rule controlling cross-origin requests
JWT	Signed session token containing user identity and role
LLM	Language model used for generation and evaluation reasoning
OCR	Conversion of image/scanned writing into machine-readable text
Rubric	Criteria and mark distribution used to grade an answer
Semantic similarity	Measure of meaning-relatedness between texts
Bloom level	Cognitive depth category used for learning analytics
Teacher override	Educator-approved final mark replacing an AI suggestion
Background job	Long AI task executed asynchronously and polled for progress
Cold start	Delay while a sleeping cloud service restarts
Supabase service role	Privileged server-only credential for storage/database operations

27. Demonstration Checklist

- Open Render /health first so the free API is awake.
- Start the local AI service and confirm localhost:8000/health.
- Keep Ollama open and verify both configured models are installed.
- Use a verified teacher account to show paper creation and publishing.
- Use a verified student account to upload a short, clearly scanned answer script.
- Show job progress and explain that AI runs locally.
- Open teacher review, compare AI and extracted answer, save a comment/override.
- Return to the student account and show final feedback, analytics, and PDF.
- Explain limitations honestly and emphasize human review.
- Never display environment secrets during the demonstration.

28. References and Source Basis

[1] TwinEval AI source repository: https://github.com/AshwinKirbaa/TwinEval_AI

[2] Current source baseline: main branch commit 6e587f4, 31 October 2026.

[3] Project deployment guide: docs/DEPLOYMENT.md.

[4] Project schema: docs/sqlite/schema.sql.

[5] Node.js documentation: https://nodejs.org/docs/

[6] Express documentation: https://expressjs.com/

[7] PostgreSQL documentation: https://www.postgresql.org/docs/

[8] Supabase documentation: https://supabase.com/docs

[9] Vercel documentation: https://vercel.com/docs

[10] Render documentation: https://render.com/docs

[11] Google Identity Services: https://developers.google.com/identity/gsi/web

[12] Ollama documentation: https://ollama.com/

[13] PyMuPDF documentation: https://pymupdf.readthedocs.io/

[14] Sentence Transformers documentation: https://www.sbert.net/

[15] ReportLab documentation: https://docs.reportlab.com/

[16] B. S. Bloom et al., Taxonomy of Educational Objectives: The Classification of Educational Goals, 1956.

[17] S. Burrows, I. Gurevych, and B. Stein, The Eras and Trends of Automatic Short Answer Grading, International Journal of Artificial Intelligence in Education, 2015.

Report integrity statement

This report is based on the current repository and the deployed architecture configured in the project. It distinguishes implemented behavior from recommended future work and does not claim unmeasured grading accuracy.